Error correction and circuit optimisations: the essentials

1 · Error correction, in five lines

- **Why it's needed.** Real qubits fail about once per 1,000 operations; Shor needs 10^8–10^{10}. Errors must be caught while the computation runs.
- **Logical qubit.** One protected qubit is stored in many physical ones. Repeated **parity checks** ("are qubits 1 and 2 equal?") reveal *where* an error happened without revealing the data; a classical **decoder** works out the correction.
- **Surface code.** A d × d grid: every ~1 µs each square measures the parity of its 4 corners. An error lights the squares at its ends; corrupting the data needs d errors in a line across the grid. Below ~1% physical error, each step d → d + 2 divides the logical error by ~10. Shor needs **d ≈ 27 → about 1,500 physical qubits per logical qubit** (≈ $2d^2$).
- **Cheap vs expensive gates.** H, S and CNOT (the *Clifford* gates) are cheap on the surface code. **T is expensive**: no code makes every gate cheap (Eastin–Knill), so each T consumes a **magic state** made in a "factory".
- **So the cost unit is the T gate (or the Toffoli).** Runtime ≈ number of Toffolis × time per Toffoli.

2 · The cost model

- **Toffoli** = reversible AND, c ⊕= a·b: **7 T**, or **4 T** if its target starts at 0 (a *temporary AND*).
- **Small rotation** (any angle not a multiple of π/4) must be approximated from H and T: **60–100 T ≈ ten Toffolis**.
- **Scratch must be uncomputed.** Leftover scratch that depends on the input destroys the interference, so: compute, copy the answer, run backwards. About **2× the cost**.
- **Rule:** build arithmetic from Toffolis, avoid rotations, and count Toffolis.

3 · Factoring optimisations: how each works → what it bought

- **Ripple-carry adder** (Cuccaro): add with Toffolis and CNOTs (MAJ blocks carry up, UMA blocks write the sum on the way down) instead of Fourier phases → **no rotations: T −95% (n = 4) to −99% (n = 12)**.
- **Semiclassical QFT**: measure each counting qubit as soon as it is done, apply the QFT's rotations classically, reuse one qubit → **qubits 4n + 2 → 2n + 3 (−41% at N = 21, −50% at size)**.
- **Control placement**: in a controlled adder, only the blocks that don't cancel need the control → **CNOTs per adder −92%**; controlled MAJ/UMA: 6n → 2n controlled gates.
- **Precomputed powers**: a^(2^j) mod N is computed on a laptop, so each rung is one multiplication → t rungs, not $2^t - 1$.
- **Temporary AND**: compute an AND into a fresh 0 for 4 T; undo it by measuring in the X basis plus a classically controlled CZ (0 T) → **14 T → 4 per pair (−71%)**.
- **Windowing**: use w bits as a table address; look up the precomputed multiple and add once → **n additions become n/w**; at N = 255, w = 4: additions −75%, depth −70%.
- **Unary iteration**: build the table lookup from a tree of ANDs → **lookup qubits 59 → 31 (−47%)**.
- **Measured uncomputation**: erase a looked-up value by X-measurement and fix the leftover sign with a cheap "phase lookup" (~√L) → **clean-up 4,094 → 157 Toffolis (−96%)**.
- **Nested windows** (window the exponent too) → multiplications **8 → 4**. **Coset representation** (pad the register so a plain adder acts mod N, small error) → **Toffolis −67 to −74%**. **Approximate QFT** (drop the tiniest rotations) → **rotations −34%**.
- **All together at N = 15: 978,000 T → 42,600 T (−96%).**

4 · Elliptic curves: the extra cost is a division

- A point addition needs the slope λ = $(y_2 - y_1)/(x_2 - x_1)$ mod p: a **modular inversion** (extended Euclid, done reversibly). It runs twice, once to make λ and once to clear it: **80% of the cost**.
- **Toy sizes, level by level:** temporary AND **−60% T**; Gidney's adder **−51%**; pseudo-Mersenne reduction (p = 2^{256} − small f, so "subtract p" = clear a bit, add f) **−84%**; **Euclidean dialog** (record 2 decision bits per GCD round, replay them on any pair) **−54% Toffolis, −43% qubits**; Jacobian coordinates (no division, but garbage) **−45%**; 16-bit windows **−93% point additions**.

5 · secp256k1 at full size (the repo, n = 256, one windowed point addition)

- **Measured uncomputation of lookups:** 1.31M → 197k Toffolis (**−85%**).
- **Conditionally inverted GCD** (IonQ): keep both GCD values odd so each step is one plain adder; multiplication 1.96M → 0.72M (**−63%**).
- **Dedicated squarer + pseudo-Mersenne fold:** 1.05M → 79k (**−92%**). → Together: **6.28M → 1.72M per addition (−73%)**.

- **"Carry is the flag" adder** (IonQ): the adder's own carry decides the reduction, then is measured away; replay step 712 → 352 (**−51%**).
- **Signed windows:** digits ±d, half-size tables (**−7%**). **SELECT-SWAP:** load several table words at once on idle qubits (**−1.2%**).
- **Peak chasing:** free scratch at whichever step sets the qubit peak → **2,233 → 1,457 qubits (−35%)**, no extra Toffoli.
- **Result: 1.31M Toffolis per addition on 1,457 qubits, 5.9% below IonQ**; whole ECDLP-256 **37.1M Toffolis (−83%)**. Trade-offs: 1,246 qubits (+65% Toffolis); 851 qubits (85.2M per addition); depth −74% with carry-lookahead adders (4.6× Toffolis).

The three ideas behind almost everything

1. **Remove rotations:** build arithmetic from Toffolis.
2. **Clean up by measuring:** an X-basis measurement reveals nothing about the data; fix the known sign it leaves (temporary AND, measured uncomputation, carry-is-flag).
3. **Do fewer, bigger steps:** one counting qubit, windowing and table lookups.